Unidad 2

Lenguajes de programación

1. **Introducción** 51
2. **Lenguajes de alto nivel** 55
3. **Lenguajes compilados** 55
4. **Lenguajes interpretados** 56
5. **Lenguajes de programación declarativos** 56

5.1. Características de los lenguajes de programación declarativos 57

5.2 Desventajas de la programación declarativa 57

5.3 Ventajas de la programación declarativa 57

6. **Programación lógica** 58
7. **Programación orientada a bases de datos** 58

7.1. Tipos de bases de datos 59

7.2 Clasificación de bases de datos según contenido 59

7.3 Modelos de bases de datos 60

8. **Lenguajes de programación imperativos** 61

8.1 Lenguajes imperativos procedurales 62

9. **Lenguajes de programación orientados a objetos** 63

9.1 Conceptos básicos de la programación orientada a objetos 64

9.2 Características de la programación orientada a objetos 65

9.3. Principales diferencias entre la programación imperativa y la programación orientada a objetos 66

Unidad 2	**Lenguajes de programación**
Expectativas de logro	**Contenidos**
1. Identifica los tipos de programación para definir su uso en la búsqueda de alternativas para desarrollar una aplicación. 2. Analizar problemas utilizando una metodología con pasos ordenados. 3. Comprenden la importancia y conveniencia de emplear una metodología para analizar, diseñar y resolver problemas.	1. Introducción 2. Lenguajes de alto nivel 3. Lenguajes compilados 4. Lenguajes interpretados 5. Lenguajes de programación declarativos 5.1. Características de los lenguajes de programación declarativos 5.2 Desventajas de la programación declarativa 5.3 Ventajas de la programación declarativa 6. Programación lógica 7. Programación orientada a bases de datos 7.1. Tipos de bases de datos 7.2 Clasificación de bases de datos según contenido 7.3 Modelos de bases de datos 8. Lenguajes de programación imperativos 8.1 Lenguajes imperativos procedurales 9. Lenguajes de programación orientados a objetos 9.1 Conceptos básicos de la programación orientada a objetos 9.2 Características de la programación orientada a objetos 9.3. Principales diferencias entre la programación imperativa y la programación orientada a objetos

Procesos y actividades sugeridas
1. Conceptualizan los términos: *Programación* y *Lenguaje de Programación*.
2. Definir la importancia de la programación en la solución de un problema.

1. Introducción

Decir que un problema se puede resolver algorítmicamente implica, informalmente, que es posible escribir un programa de computadora que producirá la respuesta correcta para cualquier entrada si permitimos que se ejecute durante el tiempo suficiente y le proporcionamos todo el espacio de almacenamiento que necesite. En la década de 1930, antes de la llegada de las computadoras, los matemáticos trabajaron con gran celo para formalizar y estudiar el concepto de algoritmo, que entonces se definía de manera informal como un conjunto claramente especificado de instrucciones sencillas a seguir para resolver un problema o calcular una función. Se idearon e investigaron varios modelos de cómputo formales.

Muchos de los primeros trabajos en este campo, llamado teoría de la computabilidad, hicieron hincapié en describir o caracterizar los problemas que se podían resolver algorítmicamente, y en presentar algunos problemas que no se podían resolver de esa manera. Uno de los resultados negativos importantes, establecido por Alan Turing, fue la demostración de la insolubilidad del "problema del paro".

Este problema consiste en determinar si cualquier algoritmo (o programa de computadora) determinado llegará en algún momento a un punto en el que se detendrá (en vez de, digamos, entrar en un ciclo infinito) al trabajar con una entrada dada. No puede existir un programa de computadora que resuelva este problema. Aunque la teoría de la computabilidad tiene implicaciones obvias y fundamentales para las ciencias de la computación, el saber que en teoría un problema se puede resolver con una computadora no basta para decirnos si resulta práctico hacerlo o no.

Por ejemplo, se podría escribir un programa perfecto para jugar ajedrez. La tarea no sería demasiado difícil; las formas de acomodar las piezas de ajedrez en el tablero son finitas, y bajo ciertas reglas una partida debe terminar después de un número finito de movimientos. El programa podría considerar todos los movimientos posibles que podría efectuar la computadora, cada una de las posibles respuestas del oponente, cada una de sus posibles respuestas a esos movimientos, y así hasta que cada una de las sucesiones de posibles movimientos llegara a su fin. Entonces, dado que la computadora conoce el resultado final de cada movimiento, podrá escoger la mejor. Según algunas estimaciones, el número de acomodos distintos de las piezas en el tablero que es razonable considerar (mucho menor que el número de sucesiones de movimientos) es de aproximadamente 1050. Un programa que las examinara todas tardaría varios miles de años en ejecutarse. Es por ello que no se ha ejecutado un programa semejante.

ALAN MATHISON TURING

Nació en Londres, 23 de junio de 1912-Wilmslow, fue un matemático, lógico, científico de la computación. Es considerado uno de los padres de la **ciencia de la computación y precursor de la informática moderna.** Proporcionó una influyente formalización de los conceptos de algoritmo y computación: **la máquina de Turing.** Formuló su propia versión de la hoy ampliamente aceptada tesis de **Church-Turing.** Durante la Segunda Guerra Mundial, trabajó en descifrar los códigos nazis, particularmente los de la máquina Enigma, y durante un tiempo fue el director de la sección Naval Enigma de Bletchley Park. Tras la guerra diseñó uno de los primeros computadores electrónicos programables digitales en el Laboratorio Nacional de Física del Reino Unido y poco tiempo después construyó otra de las primeras máquinas en la Universidad de Mánchester.

En el campo de la inteligencia artificial es conocido sobre todo por la concepción del test de Turing, un criterio según el cual puede juzgarse la inteligencia de una máquina si sus respuestas en la prueba son indistinguibles de las de un ser humano. La carrera de Turing terminó súbitamente después de ser procesado por ser homosexual. Dos años después de su condena murió, según la versión oficial por suicidio, pero su muerte ha dado lugar a otras hipótesis incluidas la del asesinato.

El 24 de diciembre de 2013, la reina Isabel II promulgó el edicto por el que se exoneró oficialmente al matemático, quedando anulados todos los cargos en su contra.

Es posible resolver una gran cantidad de problemas con aplicaciones prácticas —es decir, se pueden escribir programas para hacerlo— pero las necesidades de tiempo y almacenamiento son demasiado grandes para que tales programas tengan utilidad práctica.
Es evidente que las necesidades de tiempo y espacio de los programas son importantes en la práctica; por ello, se han convertido en el tema de estudios teóricos en el área de las ciencias de la computación llamada complejidad computacional. Una rama de tales estudios, que no se cubrirá en este libro, se ocupa de establecer una teoría formal y un tanto abstracta de la complejidad de las funciones computables. (Resolver un problema equivale a calcular una función que a partir del conjunto de entradas proporcione el conjunto de salidas.)
Se han formulado axiomas para medir la complejidad, éstos son básicos y lo bastante generales como para poder usar el número de instrucciones ejecutadas o bien el número de bits de almacenamiento que ocupa un programa como medida de su complejidad. Utilizando esos axiomas, podemos demostrar la existencia de problemas arbitrariamente complejos y de problemas para los que no existe un programa óptimo.

GARRY KASPAROV VS. DEEP BLUE:

CUANDO LA COMPUTADORA

DERROTÓ AL NÚMERO 1 DEL AJEDREZ

Escrito por: Redacción Lucidez/www.lucidez.pe

Garry Kasparov, es natural de Azerbaiyán, pero se le identifica como ruso, tiene 52 años está dedicado a la política, a la escritura y siempre será recordado como un gran maestro del ajedrez a nivel internacional que paseo su talento en el llamado deporte ciencia, por todo el mundo desde 1985 hasta el año 2000.

Pero Garry Kasparov, tuvo un reto que no pudo sortear y fue exactamente ante un rival con "inteligencia no humana", nos referimos a una computadora de la empresa norteamericana IBM bautizada como Deep Blue. El reto para probar esta computadora personal y de las más modernas para el año de 1989, era que pudiese derrotar a Garry Kasparov, en el tablero de ajedrez y así en su primera competencia con un resultado de 2-0 a favor del conocido "Maestro" ruso.

Pero IBM no se quedaría de manos cruzadas y siete años más tarde la supercomputadora de IBM Deep Blue, dio vuelta al resultado y venció a Garry Kasparov, en una nueva competición; sin embargo, Kaspárov reaccionó y terminó ganando a Deep Blue por 4-2, esto ocurrió en 1996.

Hay que señalar que Deep Blue, podía calcular 100 millones de posiciones por segundo, pero Garry Kasparov tenía su "maestría" y hoja de oficio en el ajedrez que avalaban sus victorias. A todo genio o gran competidor le llega su batalla de Waterloo, y pasó con Garry Kasparov en 1997, cuando IBM, presentó una versión mejorada de Deep Blue, la cual derrotó al ruso por 3.5 a 2.5 en una competición a seis partidas, vista a nivel mundial, por los entonces televisores analógicos.

Garry Kasparov por supuesto se defendió de las críticas y su propio descontento argumentando un cansancio, al cual la máquina no estaba sometida. Luego de 18 años muchas preguntas y dudas sobre la forma en que compitió la computadora de IBM surgieron, pero no fueron aclaradas, estas eran: ¿Ganó realmente la máquina a Garry Kasparov? ¿Existió intervención humana en las jugadas de Deep Blue? ¿O perdió Garry Kasparov debido al cansancio y otros factores emocionales?

Hay que recordar que entre 1989 y 1997 en que se produjo la disputa sobre el tablero, se iniciaba el fin de la Guerra Fría, que tenía al mundo dividido entre la Unión Soviética, que avalaba a Garry Kasparov y los Estados Unidos que tenían la obligación de mantener la imagen de primera potencia mundial, respaldados por la carrera tecnológica rol en que la IBM se metió, como una de las empresas de mayor desarrollo de los años 80 y 90.

Hay que señalar que según Wikipedia: Deep Blue era el 259º superordenador más potente, capaz de calcular 11,38 gigaflops. Ahora 18 años después del hecho en que la computadora derrotó al hombre, hay que ser francos en decir que hoy esa computadora sería un juguete de escuela primaria, pero Garry Kasparov, está en la lista de los "Top Ten", ajedrecistas históricos, ha escrito cerca de 20 libros y valgan verdades, sin el intelecto humano, la curiosidad y la sensación de querer sobrepasar fronteras la computadora o la inteligencia artificial, simplemente no existiría.

Actividad

Escribe un ensayo sobre este tema: Las computadoras han superado al hombre.

Presenta este ensayo a tu maestro.

1.2 Tipos de lenguajes de programación

En esta unidad profundizaremos un poco más sobre los diferentes tipos de lenguajes de programación sus usos más comunes especialmente los lenguajes de alto nivel.

Entre los lenguajes de programación tenemos:

1. Lenguaje de máquina
2. Lenguaje ensamblador
3. Lenguajes de alto nivel.

2. Lenguajes de alto nivel

Los primeros programas ensambladores producían sólo una instrucción en lenguaje de máquina por cada instrucción del programa fuente. Para agilizar la codificación, se desarrollaron programas ensambladores que podían producir una cantidad variable de instrucciones en lenguaje de máquina por cada instrucción del programa fuente. Dicho de otra manera, una sola macroinstrucción podía producir varias líneas de código en lenguaje de máquina. Por ejemplo, el programador podría escribir "LEER ARCHIVO", y el programa traductor produciría una serie detallada de instrucciones al lenguaje de máquina previamente preparadas, con lo que se copiaría un registro del archivo que estuviera leyendo el dispositivo de entrada a la memoria principal. Así, el programador no se tenía que ocupar de escribir una instrucción por cada operación de máquina realizada.

El desarrollo de las técnicas nemotécnicas y las macroinstrucciones condujo, a su vez, al desarrollo de lenguajes de alto nivel que a menudo están orientados hacia una clase determinada de problemas de proceso. Por ejemplo, se han diseñado varios lenguajes para procesar problemas científico-matemático, asimismo han aparecido otros lenguajes que hacen hincapié en las aplicaciones de proceso de archivos.

A diferencia de los programas de ensamble, los programas en lenguaje de alto nivel se pueden utilizar con diferentes marcas de computadores sin tener que hacer modificaciones considerables. Esto permite reducir sustancialmente el costo de la reprogramación cuando se adquiere equipo nuevo. Otras ventajas de los lenguajes de alto nivel son:

- Son más fáciles de aprender que los lenguajes ensambladores.
- Se pueden escribir más rápidamente.
- Permiten tener mejor documentación.
- Son más fáciles de mantener.
- Un programador que sepa escribir programas en uno de estos lenguajes no está limitado a utilizar un solo tipo de máquina.

3. Lenguajes compilados

Naturalmente, un programa que se escribe en un lenguaje de alto nivel también tiene que traducirse a un código que pueda utilizar la máquina. Los programas traductores que pueden realizar esta operación se llaman compiladores. Éstos, como los programas ensambladores avanzados, pueden generar muchas líneas de código de máquina por cada proposición del programa fuente. Se requiere una corrida de compilación antes de procesar los datos de un problema.

Los compiladores son aquellos cuya función es traducir un programa escrito en un determinado lenguaje a un idioma que la computadora entienda (lenguaje máquina con código binario).

Al usar un lenguaje compilado (como lo son los lenguajes del popular Visual Studio de Microsoft), el programa desarrollado nunca se ejecuta mientras haya errores, sino hasta que luego de haber compilado el programa, ya no aparecen errores en el código.

4. Lenguajes interpretados

Se puede también utilizar una alternativa diferente de los compiladores para traducir lenguajes de alto nivel. En vez de traducir el programa fuente y grabar en forma permanente el código objeto que se produce durante la corrida de compilación para utilizarlo en una corrida de producción futura, el programador sólo carga el programa fuente en la computadora junto con los datos que se van a procesar. A continuación, un programa intérprete, almacenado en el sistema operativo del disco, o incluido de manera permanente dentro de la máquina, convierte cada proposición del programa fuente en lenguaje de máquina conforme vaya siendo necesario durante el proceso de los datos. No se graba el código objeto para utilizarlo posteriormente.

La siguiente vez que se utilice una instrucción, se le debe interpretar otra vez y traducir a lenguaje máquina. Por ejemplo, durante el procesamiento repetitivo de los pasos de un ciclo, cada instrucción del ciclo tendrá que volver a ser interpretado cada vez que se ejecute el ciclo, lo cual hace que el programa sea más lento en tiempo de ejecución (porque se va revisando el código en tiempo de ejecución) pero más rápido en tiempo de diseño (porque no se tiene que estar compilando a cada momento el código completo).

El intérprete elimina la necesidad de realizar una corrida de compilación después de cada modificación del programa cuando se quiere agregar funciones o corregir errores; pero es obvio que un programa objeto compilado con antelación deberá ejecutarse con mucha mayor rapidez que uno que se debe interpretar a cada paso durante una corrida de producción.

5. Lenguajes de programación declarativos

Se les conoce como lenguajes declarativos en ciencias computacionales a aquellos lenguajes de programación en los cuales se le indica a la computadora qué es lo que se desea obtener o qué es lo que se esta buscando, por ejemplo: Obtener los nombres de todos los empleados que tengan más de 32 años. Eso se puede lograr con un lenguaje declarativo como SQL. La programación declarativa es una forma de programación que implica la descripción de un problema dado en lugar de proveer una solución para dicho problema, dejando la interpretación de los pasos específicos para llegar a dicha solución a un intérprete no especificado. La programación declarativa adopta, por lo tanto, un enfoque diferente al de la programación imperativa tradicional. En otras palabras, la programación declarativa provee el "qué", pero deja el "cómo" liberado a la implementación particular del intérprete. Por lo tanto se puede ver que la programación declarativa tiene dos fases bien diferenciadas, la declaración y la interpretación.

Es importante señalar que a pesar de hacer referencia a intérprete, no hay que limitarse a "lenguajes interpretados" en el sentido habitual del término, sino que también se puede estar trabajando con "lenguajes compilados".

5.1. Características de los lenguajes de programación declarativos

- Los lenguajes declarativos están orientados a buscar la solución del problema, sin preocuparse por la forma de llegar a ello; es decir, el programador debe concentrarse en la lógica del algoritmo, más que en el control de la secuencia.
- Los programas están formados por un conjunto de definiciones o ecuaciones, las cuales describen lo que debe ser calculado, no en sí la forma de hacerlo.
- Las variables sólo pueden tener asignado un solo valor a lo largo de la ejecución del programa, lo cual implica que no puede existir asignación destructiva. Debido a esto, cobra especial importancia el uso del anidamiento y la recursividad.
- Las listas representan la estructura fundamental de datos.
- El orden de la ejecución no resulta importante debido a que no existen efectos colaterales; es decir, que al calcular un valor, resulta imposible afectar el cálculo de otros y con esto se puede afirmar que cualquier secuencia de ejecución deberá conducir al mismo resultado.
- Las expresiones o definiciones pueden ser usadas como valores y por lo tanto se pueden tratar como argumentos de otras definiciones.
- El control de la ejecución no es responsabilidad del programador.

5.2 Desventajas de la programación declarativa

La principal desventaja de la programación declarativa es que no puede resolver cualquier problema dado, sino que está restringida al subconjunto de problemas para los que el intérprete o compilador fue diseñado. Otra desventaja de la programación declarativa está relacionada con la eficiencia. Dado que es necesaria una fase de interpretación extra, en la cual se deben evaluar todas las consecuencias de todas las declaraciones realizadas, el proceso es relativamente más lento que en la programación imperativa, en que los cambios de estado del sistema están dados por instrucciones particulares y no por un conjunto de condiciones arbitrariamente grande.

5.3 Ventajas de la programación declarativa

A pesar de lo anterior existen algunas ventajas en el uso de la programación declarativa. Entre las ventajas se destaca que la solución de un problema se puede realizar con un nivel de abstracción considerablemente alto, sin entrar en detalles de implementación irrelevantes, lo que hace a las soluciones más fácil de entender por las personas. La resolución de problemas complejos es resuelta por el intérprete a partir de la declaración de las condiciones dadas. La programación declarativa es muy usada en la resolución de problemas relacionados con inteligencia artificial, bases de datos, configuración, y comunicación entre procesos; sin embargo, ningún lenguaje declarativo se aproxima en popularidad a los lenguajes imperativos.

Ejemplos de lenguajes declarativos

Algunos lenguajes declarativos que se pueden mencionar son:

• PROLOG

• SQL

• HTML

• WSDL (Web Services Description Language)

• XML Stylesheet Language for Transformation

6. Programación lógica

La idea fundamental de la programación lógica consiste en emplear la lógica como lenguaje de programación. La lógica no es imperativa porque no sirve para indicar cómo resolver un problema (órdenes). La lógica es declarativa porque sirve para especificar qué problema resolver (condiciones). En la programación lógica, se especifican las condiciones que satisfacen las soluciones, se deducen las soluciones a partir de las condiciones y el énfasis de todo está en qué problema resolver. El problema se describe especificando qué caracteriza a sus posibles soluciones.

La programación lógica, junto con la funcional, forma parte de lo que se conoce como programación declarativa. En los lenguajes tradicionales, la programación consiste en indicar cómo resolver un problema mediante sentencias; en la programación lógica, se trabaja de forma descriptiva, estableciendo relaciones entre entidades, indicando no cómo, sino qué hacer. Se establece entonces que la idea esencial de la programación lógica es: algoritmos = lógica + control. Es decir, un algoritmo se construye especificando conocimiento en un lenguaje formal (lógica de primer orden), y el problema se resuelve mediante un mecanismo de inferencia (control) que actúa sobre aquél.

Al hacer un recorrido por la programación lógica, aparece como uno de sus lenguajes más representativos, Prolog, que es un clásico de la inteligencia artificial y que se aplica de múltiples formas en el desarrollo de software comercial.

7. Programación orientada a bases de datos

Las bases de datos son programas que administran información y hacen más ordenada la información, aparte de hacerla fácil de buscar y por supuesto de encontrar. Las características de las bases de datos pueden ser ventajosas o desventajosas: pueden ayudar a almacenar, organizar, recuperar, comunicar alguna manera ya que existen enormes cantidades de información en bases de datos de las que no se tiene control del acceso.

Las bases de datos tienen muchos usos: facilitan el almacenamiento de grandes cantidades de información; permiten la recuperación rápida y flexible de información, con ellas se puede organizar y reorganizar la información, así como imprimirla o distribuirla en formas diversas. Es claro que los lenguajes orientados a bases de datos son declarativos y no imperativos, pues el problema es "qué" se quiere hacer o "qué" se necesita buscar y encontrar en la base de datos, y no se enfatiza el "cómo".

Una base de datos también se puede definir como un banco de datos o conjunto de datos que pertenecen al mismo contexto, almacenados sistemáticamente para su posterior uso. En este sentido, una biblioteca puede considerarse una base de datos compuesta en su mayoría por documentos y textos impresos en papel e indexados para su consulta.

En la actualidad, y gracias al desarrollo tecnológico de campos como la informática y la electrónica, la mayoría de las bases de datos tienen formato electrónico, que ofrece un amplio rango de soluciones al problema de almacenar datos. Los sistemas gestores de bases de datos (SGBD) permiten almacenar y posteriormente acceder a los datos de forma rápida y estructurada.

7.1. Tipos de bases de datos

Las bases de datos pueden clasificarse de varias maneras, de acuerdo al criterio elegido para su clasificación, que puede ser según la variabilidad de los datos almacenados o según el contenido.

Clasificación de bases de datos según la variabilidad de los datos almacenados

Esta clasificación depende de cómo los datos cambian o varían dentro de la base de datos, o si se mantienen estáticos sin sufrir modificaciones o cambios. Dentro de esta clasificación se tiene lo siguiente:

1. **Bases de datos estáticas:** Éstas son bases de datos de sólo lectura, utilizadas primordialmente para almacenar datos históricos que posteriormente se pueden utilizar para estudiar el comportamiento de un conjunto de datos a través del tiempo, realizar proyecciones y tomar decisiones.

2. **Bases de datos dinámicas:** Éstas son bases de datos donde la información almacenada se modifica con el tiempo, permitiendo operaciones como actualización y adición de datos, además de las operaciones fundamentales de consulta. Un ejemplo de esto puede ser la base de datos utilizada en un sistema de información de una tienda de abarrotes, una farmacia, un videoclub, etc.

7.2 Clasificación de bases de datos según contenido

Esta clasificación basa su enfoque en el contenido o lo que almacena la base de datos, de esta manera:

1. **Bases de datos bibliográficas:** Solo contienen un "representante" de la fuente primaria, que permite localizarla. Un registro típico de una base de datos bibliográfica contiene información sobre el autor, fecha de publicación, editorial, título, edición, de una determinada publicación, etc. Puede contener un resumen o extracto de la publicación original, pero nunca el texto completo, porque sino se estaría en presencia de una base de datos a texto completo (o de fuentes primarias). Como su nombre lo indica, el contenido son cifras o números. Por ejemplo, una colección de resultados de análisis de laboratorio, entre otras.

2. **Bases de datos de texto completo:** Almacenan las fuentes primarias, como por ejemplo, todo el contenido de todas las ediciones de una colección de revistas científicas.

3. **Directorios:** Un ejemplo son las guías telefónicas en formato electrónico.

4. **Banco de imágenes, audio, multimedia, etc.**

7.3 Modelos de bases de datos

Además de la clasificación por la función de las bases de datos, éstas también se pueden clasificar de acuerdo a su modelo de administración de datos. Un modelo de datos es básicamente una "descripción" de algo conocido como contenedor de datos (algo en donde se guarda la información), así como de los métodos para almacenar y recuperar información de esos contenedores. Los modelos de datos no son cosas físicas: son abstracciones que permiten la implementación de un sistema eficiente de base de datos; por lo general se refieren a algoritmos, y conceptos matemáticos.

Algunos modelos con frecuencia utilizados en las bases de datos son:

1. **Bases de datos jerárquicas:** Éstas son bases de datos que, como su nombre indica, almacenan su información en una estructura jerárquica. En este modelo los datos se organizan en una forma similar a un árbol (visto al revés), en donde un nodo padre de información puede tener varios hijos. El nodo que no tiene padres es llamado raíz, y a los nodos que no tienen hijos se los conoce como hojas. Las bases de datos jerárquicas son especialmente útiles en el caso de aplicaciones que manejan un gran volumen de información y datos muy compartidos permitiendo crear estructuras estables y de gran rendimiento. Una de las principales limitaciones de este modelo es su incapacidad de representar eficientemente la redundancia de datos.

2. **Bases de datos de red:** Éste es un modelo ligeramente distinto del jerárquico; su diferencia fundamental es la modificación del concepto de nodo: se permite que un mismo nodo tenga varios padres (posibilidad no permitida en el modelo jerárquico). Fue una gran mejora con respecto al modelo jerárquico, ya que ofrecía una solución eficiente al problema de redundancia de datos; pero, aun así, la dificultad que significa administrar la información en una base de datos de red ha significado que sea un modelo utilizado en su mayoría por programadores más que por usuarios finales.

3. **Bases de datos relacional:** Éste es el modelo más utilizado en la actualidad para modelar problemas reales y administrar datos dinámicamente. Tras ser postulados sus fundamentos en 1970 por Edgar Frank Codd, de los laboratorios IBM en San José (California), no tardó en consolidarse como un nuevo paradigma en los modelos de base de datos. Su idea fundamental es el uso de "relaciones". Estas relaciones podrían considerarse en forma lógica como conjuntos de datos llamados "tuplas". Pese a que ésta es la teoría de las bases de datos relacionales creadas por Edgar Frank Codd, la mayoría de las veces se conceptualiza de una manera más fácil de imaginar. Esto es pensando en cada relación como si fuese una tabla que está compuesta por registros (las filas de una tabla), que representarían las tuplas, y campos (las columnas de una tabla).

En este modelo, el lugar y la forma en que se almacenen los datos no tienen relevancia (a diferencia de otros modelos como el jerárquico y el de red). Esto tiene la considerable ventaja de que es más fácil de entender y de utilizar para un usuario esporádico de la base de datos. La información puede ser recuperada o almacenada mediante "consultas" que ofrecen una amplia flexibilidad y poder para administrar la información.

El lenguaje más habitual para construir las consultas a bases de datos relacionales es SQL, Structured Query Language o Lenguaje Estructurado de Consultas, un estándar implementado por los principales motores o sistemas de gestión de bases de datos relacionales. Durante su diseño, una base de datos relacional pasa por un proceso al que se le conoce como normalización (reglas fundamentales para el buen funcionamiento de una base de datos relacional) de una base de datos. Durante los años '80 (1980-1989) la aparición de dBASE produjo una revolución en los lenguajes de programación y sistemas de administración de datos. Aunque nunca debe olvidarse que dBase no utilizaba SQL como lenguaje base para su gestión.

4. **Bases de datos orientadas a objetos:** este modelo, bastante reciente, y propio de los modelos informáticos orientados a objetos, trata de almacenar en la base de datos los objetos completos (estado y comportamiento). Una base de datos orientada a objetos es una base de datos que incorpora todos los conceptos importantes del paradigma de objetos: encapsulación (propiedad que permite ocultar la información al resto de los objetos, impidiendo así accesos incorrectos o conflictos), herencia (propiedad a través de la cual los objetos heredan comportamiento dentro de una jerarquía de clases y polimorfismo (propiedad de una operación mediante la cual puede ser aplicada a distintos tipos de objetos. En bases de datos orientadas a objetos, los usuarios pueden definir operaciones sobre los datos como parte de la definición de la base de datos.

5. **Bases de datos documentales:** permiten la indexación a texto completo, y en líneas generales realizar búsquedas más potentes.

6. **Base de datos deductivas:** un sistema de base de datos deductivas, es un sistema de base de datos pero con la diferencia de que permite hacer deducciones a través de inferencias. Se basa principalmente en reglas y hechos que son almacenados en la base de datos. También las bases de datos deductivas son llamadas base de datos lógica, a raíz de que se basan en lógica matemática.

8. Lenguajes de programación imperativos

En ciencias de la computación se llama lenguajes imperativos a aquellos en los cuales se le ordena a la computadora cómo realizar una tarea siguiendo una serie de pasos o instrucciones, por ejemplo:

Paso 1, solicitar número.

Paso 2, multiplicar número por dos.

Paso 3, imprimir resultado de la operación.

Paso 4, etc,

El proceso anterior se puede realizar con un lenguaje imperativo como por ejemplo BASIC, C, C++, Java, Clipper, Dbase, C#, PHP, Perl, etc. Dentro de la programación imperativa, se tiene un conjunto de instrucciones que le indican a la computadora cómo realizar una tarea.

Los lenguajes imperativos se basan en comandos u órdenes que se le dan a la computadora para que haga algo, con el fin de organizar o cambiar valores en ciertas partes de la memoria. La ejecución de estos comandos se realiza, en la mayor parte de ellos, secuencialmente, es decir, hasta que un comando no ha sido ejecutado no se lee el siguiente.

Según el dominio, o mejor dicho con el propósito que se utiliza el programa, se puede hablar de lenguajes de dominio específico y de dominio general.

8.1 Lenguajes imperativos procedurales

En los lenguajes tradicionales o procedurales, es la aplicación quien controla qué porciones de código se ejecuta, y la secuencia en que este se ejecuta. La ejecución de la aplicación se inicia con la primera línea de código, y sigue una ruta predefinida a través de la aplicación, llamando procedimientos según sea necesario.

Los lenguajes procedurales están fundamentados en la utilización de variables para almacenar valores y en la realización de operaciones con los datos almacenados. Algunos ejemplos son: FORTRAN, PASCAL, C, ADA, ALGOL,...

En este tipo de lenguajes, la arquitectura consta de una secuencia de celdas, llamadas memoria, en las cuales se pueden guardar en forma codificada, lo mismo datos que instrucciones; y de un procesador, el cual es capaz de ejecutar

de manera secuencial una serie de operaciones, principalmente aritméticas y booleanas, llamadas comandos. En general, un lenguaje procedural ofrece al programador conceptos que se traducen de forma natural al modelo de la máquina. El programador tiene que traducir la solución abstracta del problema a términos muy primitivos, cercanos a la máquina.

Con un lenguaje procedural el usuario (normalmente será un programador) especifica qué datos se necesitan y cómo obtenerlos. Esto quiere decir que el usuario debe especificar todas las operaciones de acceso a datos llamando a los procedimientos necesarios para obtener la información requerida. Estos lenguajes acceden a un registro, lo procesan y basándose en los resultados obtenidos, acceden a otro registro, que también deben procesar. Así se va accediendo a registros y se van procesando hasta que se obtienen los datos deseados. Las sentencias de un lenguaje procedural deben estar embebidas en un lenguaje de alto nivel, ya que se necesitan sus estructuras (bucles, condicionales, etc.) para obtener y procesar cada registro individual.

Algunos lenguajes imperativos

Algunos lenguajes de programación imperativos que se pueden mencionar son:

- Visual BASIC
- C
- C++
- Java
- C#
- PHP

Diferencia entre lenguajes declarativos e imperativos

En los lenguajes declarativos las sentencias que se utilizan lo que hacen es describir el problema que se quiere solucionar, pero no las instrucciones necesarias para solucionarlo. Esto último se realizará mediante mecanismos internos de inferencia de información a partir de la descripción realizada.

Los lenguajes imperativos describen paso a paso un conjunto de instrucciones que deben ejecutarse para variar el estado un programa y hallar la solución, es decir, un algoritmo en el que se describen los pasos necesarios para solucionar un problema.

9. Lenguajes de programación orientados a objetos

En la Programación Orientada a Objetos (POO u OOP según siglas en inglés) se definen los programas en términos de "clases de objetos", objetos que son entidades que combinan estado (es decir, datos) comportamiento (esto es, procedimientos o métodos) e identidad (propiedad del objeto que lo diferencia del resto). La programación orientada a objetos expresa un programa como un conjunto de estos objetos, que colaboran entre ellos para realizar tareas. Esto permite hacer los programas módulos más fáciles de escribir, mantener y reutilizar.

De esta forma, un objeto contiene toda la información, (los denominados atributos) que permite definirlo e identificarlo frente a otros objetos pertenecientes a otras clases (e incluso entre objetos de la misma clase, al poder tener valores bien diferenciados en sus atributos). A su vez, dispone de mecanismos de interacción (los llamados métodos) que favorecen la comunicación entre objetos (de una misma clase o de distintas), y en consecuencia, el cambio de estado en los propios objetos. Esta característica lleva a tratarlos como unidades indivisibles, en las que no se separan (ni deben separarse) información (datos) y procesamiento (métodos).

Dada esta propiedad de conjunto de una clase de objetos, que al contar con una serie de atributos definitorios, requiere de unos métodos para poder tratarlos (lo que hace que ambos conceptos están íntimamente entrelazados), el programador debe pensar indistintamente en ambos términos, ya que no debe nunca separar o dar mayor importancia a los atributos a favor de los métodos, ni viceversa. Hacerlo puede llevar al programador a seguir el hábito erróneo de crear clases contenedoras de información por un lado y clases con métodos que manejen esa información por otro (llegando a una programación estructurada camuflada en un lenguaje de programación orientada a objetos).

Esto difiere de los lenguajes imperativos tradicionales, en los que los datos y los procedimientos están separados y sin relación, ya que lo único que se busca es el procesamiento de unos datos ya que lo único que se busca es el procesamiento de unos datos de entrada para obtener otros de salida. La programación estructurada anima al programador a pensar sobre todo en términos de procedimientos o funciones, y en segundo lugar en las estructuras de datos que esos procedimientos manejan. Los programadores que de lenguajes imperativos escriben funciones y después les pasan los datos. Los programadores que emplean lenguajes orientados a objetos definen objetos con datos y métodos y después envían mensajes a los objetos diciendo que realicen esos métodos por sí mismos.

Un objeto se puede definir como un grupo de procedimientos que comparten un estado. Se define al conjunto de datos como "estado", y "métodos" como el conjunto de procedimientos que pueden alterar ese estado. Un programa orientado a objetos es un método de implementación en el que los programas están organizados como colecciones de objetos, donde cada uno es una instancia de alguna clase, y donde todas las clases son miembros de una jerarquía de clases conectadas por relaciones de herencia. Este tipo de lenguajes son muy recientes en comparación a los primeros lenguajes de programación que aparecieron.

9.1 Conceptos básicos de la programación orientada a objetos

La programación orientada a objetos es una nueva forma de programar que trata de encontrar la solución a problemas de una forma que ofrece muchas ventajas y facilidades que no se tenían anteriormente. Introduce nuevos conceptos, que superan y amplían conceptos antiguos ya conocidos. Entre ellos destacan los siguientes:

• **Objeto:** entidad provista de un conjunto de propiedades o atributos (datos) y de comportamiento o funcionalidad ("métodos"). Corresponden a los objetos reales del mundo que nos rodea, o a objetos internos del sistema (del programa).

• **Clase:** definiciones de las propiedades y comportamiento de un tipo de objeto concreto. La instanciación es la lectura de estas definiciones y la creación de un objeto a partir de ellas. Una clase es una colección de objetos similares o la implementación, declaración o definición de un tipo de objeto. Cada vez que se construye un objeto de una clase se crea una instancia de esa clase. Por ejemplo en Visual Basic, se tiene la clase Form, y se pueden crear instancias de esa clase al tener Form1, Form2, etc. Así se está creando una instancia de la clase Form.

• **Método:** algoritmo asociado a un objeto (o a una clase de objetos), cuya ejecución se desencadena tras la recepción de un "mensaje". Desde el punto de vista del comportamiento, es lo que el objeto puede hacer. Un método puede producir un cambio en lar propiedades del objeto, o la generación de un "evento" con un nuevo mensaje para otro objeto del sistema.

• **Evento:** un suceso en el sistema (tal como una interacción del usuario con la máquina, o un mensaje enviado por un objeto). El sistema maneja el evento enviando el mensaje adecuado al objeto pertinente.

• **Mensaje:** una comunicación dirigida a un objeto, que le ordena que ejecute uno de sus métodos con ciertos parámetros asociados al evento que lo generó.

• **Propiedad o atributo:** contenedor de un tipo de datos asociados a un objeto (o a una clase de objetos), que hace los datos visibles desde fuera del objeto, y cuyo valor puede ser alterado por la ejecución de algún método.

• **Estado interno:** es una propiedad invisible de los objetos, que puede ser únicamente accedida y alterada por un método del objeto, y que se utiliza para indicar distintas situaciones posibles para el objeto (o clase de objetos).

9.3. Principales diferencias entre la programación imperativa y la programación orientada a objetos

• La programación orientada a objetos es más moderna, es una evolución de la programación imperativa plasmada en el diseño de una familia de lenguajes conceptos que existían previamente, con algunos nuevos.

• La programación orientada a objetos se basa en lenguajes que soportan sintáctica y semánticamente la unión entre los tipos abstractos de datos y sus operaciones (a esta unión se la suele llamar clase).

• La programación orientada a objetos incorpora en su entorno de ejecución mecanismos tales como el polimorfismo y el envío de mensajes entre objetos.

Algunos lenguajes orientados a objetos

Entre los lenguajes orientados a objetos más importantes que se pueden mencionar, aparecen los siguientes:

- Ada
- C++
- C#
- VB.NET
- Clarion
- Delphi
- Eiffel
- Java
- Objective-C
- Ocaml
- Oz
- PHP
- PowerBuilder
- Pitón
- Ruby
- Smalltalk